ACTIVIDAD INDIVIDUAL

API REST CON EXPRESS, MYSQL

Programa de formación: Análisis y Desarrollo de Software (ADSO)

Aprendiz: Cristian David Yalanda

Ficha: 3312932 Instructor(a): Yoly Alexandra

Fecha de entrega: 26 / 10 / 2026

1. Objetivo

Desarrollar habilidades para construir y probar una API REST con Express, CORS y MySQL, utilizando una base de datos. Identificar errores de programación, explicar sus causas y proponer soluciones justificadas.

2. Situación problema

Una empresa necesita una API para administrar los productos de su inventario. El sistema debe permitir registrar productos y consultar los que se encuentran almacenados en la base de datos. Cada producto debe tener un identificador, nombre, categoría, precio y cantidad disponible. El aprendiz construirá la solución y realizará una revisión técnica de errores.

3. Fases de la actividad

Fase 1. Investigación y preparación del entorno

Investigue y explique la utilidad de los siguientes conceptos:

1. ¿Qué es Node.js y qué función cumple?..

node.js es un entorno que permite ejecutar javascript fuera del navegador. se utiliza principalmente para desarrollar aplicaciones y servidores en el backend.

en nuestro proyecto, node.js permite ejecutar el servidor que recibe las peticiones de la aplicación y se comunica con la base de datos.

2. ¿Qué es Express y por qué se utiliza para desarrollar una API?

express es un framework de javascript que funciona sobre node.js y facilita la creación de servidores y api.

se utiliza para crear las rutas de nuestra aplicación, recibir peticiones como get y post, enviar respuestas y manejar diferentes funciones del servidor de una manera más sencilla.

3. ¿Qué es una API REST?

una api rest es una forma de crear servicios que permiten que diferentes aplicaciones se comuniquen mediante peticiones http.

por ejemplo, una aplicación frontend puede hacer una petición get para consultar trabajadores o una petición post para registrar un nuevo trabajador. la api recibe la información, realiza la operación correspondiente y devuelve una respuesta

4. ¿Qué función cumple CORS?

cors permite controlar las solicitudes que se realizan entre diferentes orígenes.

en nuestro proyecto es necesario porque el frontend y el backend pueden ejecutarse en diferentes puertos. cors permite que el frontend pueda comunicarse con nuestra api sin que el navegador bloquee la petición.

5. ¿Qué es MySQL y para qué sirve una base de datos?

mysql es un sistema de gestión de bases de datos relacionales que permite almacenar y administrar información. Una base de datos sirve para guardar información de forma organizada y poder consultarla, modificarla, eliminarla o agregar nuevos registros.

en nuestro proyecto, mysql puede utilizarse para almacenar información de los trabajadores, como nombre, apellido, dirección y teléfono.

6. ¿Qué información se necesita para establecer una conexión con MySQL en línea?

para conectarse a una base de datos mysql en línea normalmente se necesita:

- host: dirección del servidor donde está la base de datos.
- usuario: usuario autorizado para acceder.
- contraseña: contraseña del usuario.
- base de datos: nombre de la base de datos que se quiere utilizar.
- puerto: puerto utilizado por mysql, normalmente 3306.

estos datos se utilizan para configurar la conexión desde node.js mediante la dependencia mysql2

7. ¿Qué función cumple cada dependencia instalada en el proyecto?

- express: permite crear el servidor, las rutas y la api.
- cors: permite las solicitudes entre diferentes orígenes, por ejemplo, entre el frontend y el backend.
- mysql2: permite conectar node.js con mysql y ejecutar consultas sobre la base de datos.
- nodemon: reinicia automáticamente el servidor cuando se realizan cambios en el código, facilitando el desarrollo.

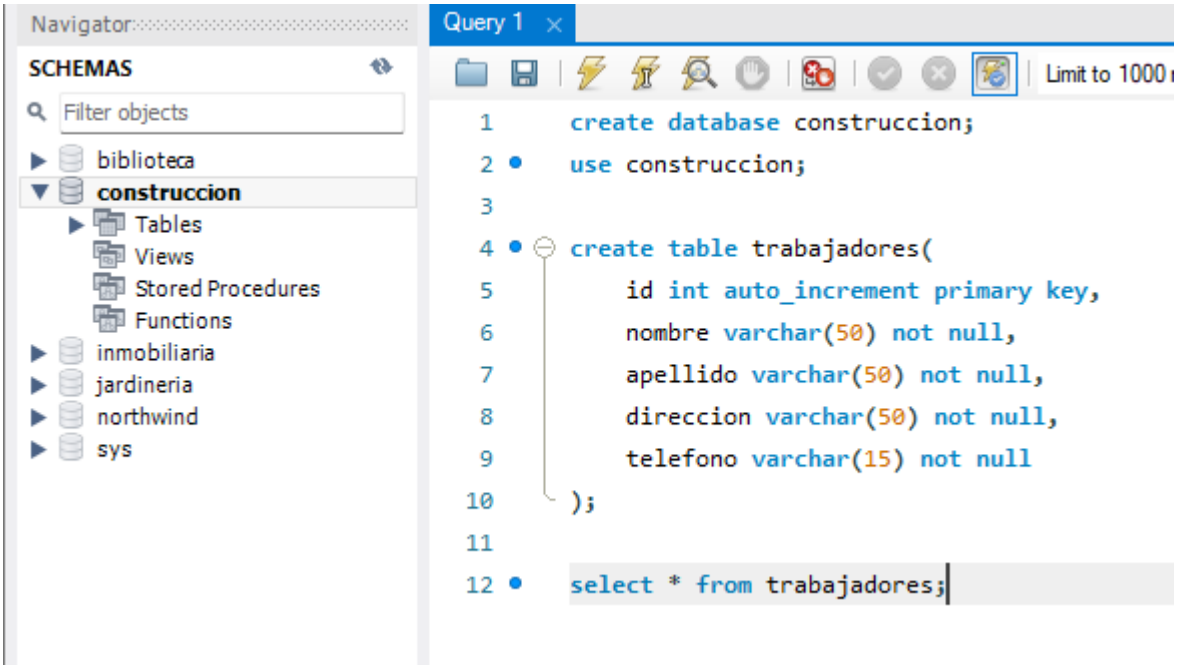
Evidencia: documento con las respuestas y explicación de la configuración del entorno.

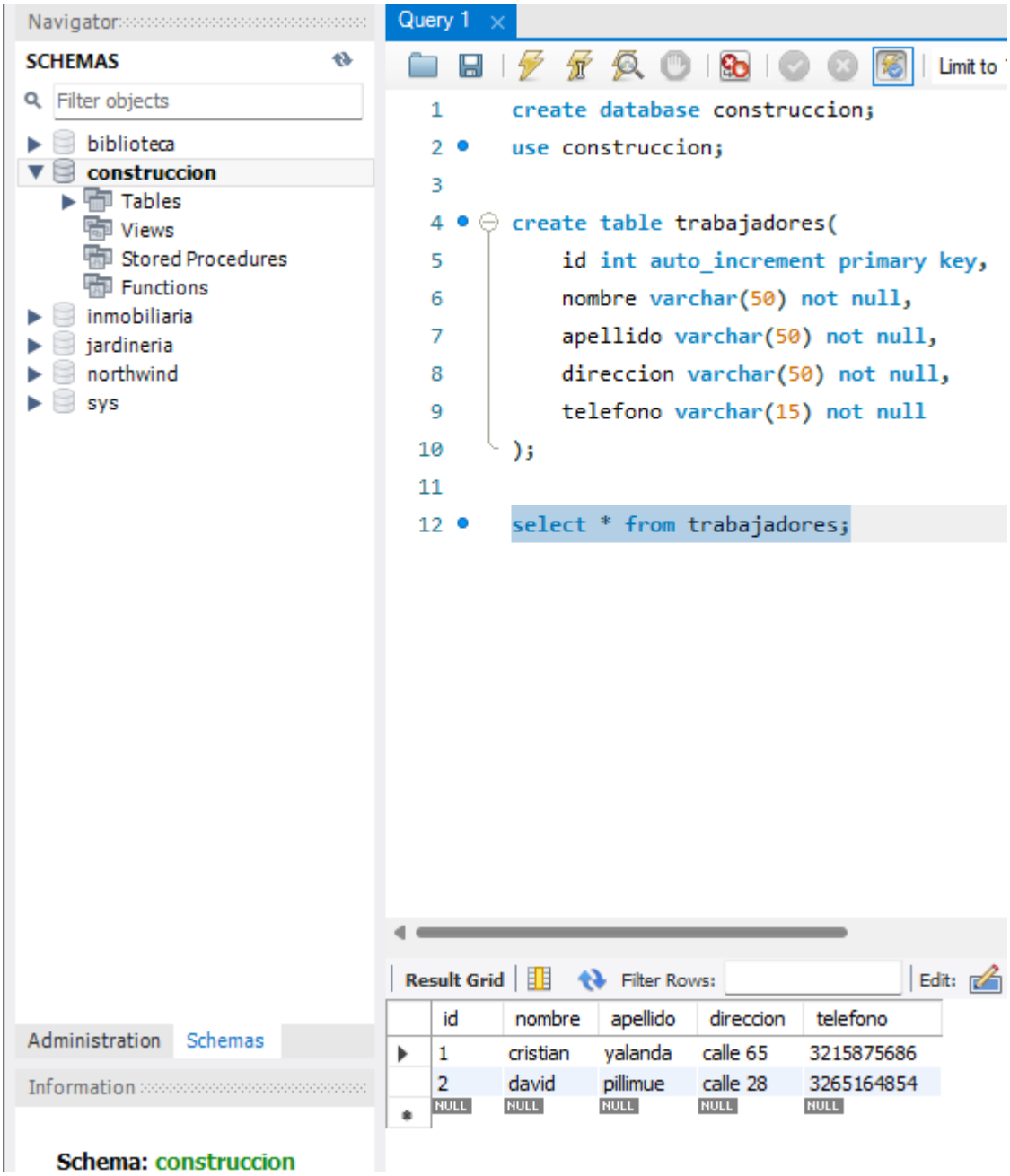
Fase 2. Diseño de la base de datos

Cree una base de datos y cree una tabla diferente a la utilizada en el ejercicio anterior, destinada a almacenar productos. La tabla debe contener: identificador único, nombre del producto, categoría, precio y cantidad disponible.

Justifique la elección del tipo de dato de cada campo y explique por qué el identificador debe ser único. Verifique la estructura de la tabla e inserte registros de prueba desde el gestor de base de datos.

Evidencia: captura de la estructura de la tabla y de los registros de prueba en MySQL.


Navigator
SCHEMAS
Filter objects
biblioteca
construccion
Tables
Views
Stored Procedures
Functions
inmobiliaria
jardineria
northwind
sys
Query 1
Limit to 1000
1 create database construccion;
2 use construccion;
3
4 create table trabajadores(
5 id int auto_increment primary key,
6 nombre varchar(50) not null,
7 apellido varchar(50) not null,
8 direccion varchar(50) not null,
9 telefono varchar(15) not null
10);
11
12 select * from trabajadores;



Fase 3. Construcción de la API

Desarrolle una API con Express que permita verificar el funcionamiento del servidor, registrar productos, consultar todos los productos almacenados, recibir y responder información en formato JSON, validar los datos antes de guardarlos y manejar errores de conexión y consultas.

Método	Ruta	Función
GET	/prueba	Verificar el funcionamiento del servidor
POST	/productos	Registrar un producto
GET	/productos	Consultar todos los productos

Explique el propósito de cada ruta, el método HTTP utilizado y el resultado esperado. Evidencia: proyecto funcional y capturas de las solicitudes realizadas en Postman.

Fase 4. Revisión y análisis de errores

El instructor suministrará una implementación inicial de una API que contiene errores. Revise el material, identifique los problemas y proponga las correcciones correspondientes. No se limite a copiar una solución: cada corrección debe estar justificada.

8. Configuración del middleware necesario para recibir solicitudes JSON.
9. Conexión con la base de datos.
10. Nombres de tablas y campos utilizados en las consultas.
11. Inserción incorrecta de registros.
12. Respuestas invertidas entre operaciones exitosas y fallidas.
13. Consulta incorrecta de los registros.
14. Validación de los datos recibidos.
15. Uso adecuado de códigos de estado HTTP.
16. Manejo de errores en las consultas.
17. Protección de las credenciales de conexión.

Para cada error encontrado, documente:

- ¿Cuál es el error?
- ¿Por qué ocurre?
- ¿Qué consecuencia tiene?
- ¿Cómo debería corregirse?
- ¿Qué prueba permite comprobar la solución?

Evidencia: informe de revisión técnica con cada error explicado y su respectiva solución.

Fase 5. Pruebas funcionales

Realice pruebas en Postman para verificar:

- Registro exitoso de un producto.
- Consulta de productos registrados.
- Envío de información incompleta.
- Envío de valores inválidos para el precio o la cantidad.
- Respuesta del servidor cuando ocurre un error.
- Persistencia de los registros después de reiniciar el servidor.

Evidencia: capturas de cada prueba, incluyendo respuestas exitosas y errores controlados.

Fase 6. Documentación y repositorio

Cree un repositorio en GitHub o GitLab que contenga:

- El proyecto funcional.
- La documentación de instalación, configuración y ejecución.
- Las instrucciones para configurar la conexión a la base de datos.
- Las evidencias de las pruebas realizadas.
- El informe de análisis de errores y conclusiones.

El repositorio no debe contener contraseñas, certificados privados ni credenciales de acceso.

4. Preguntas de análisis

18. ¿Qué diferencia existe entre GET y POST?

get se utiliza principalmente para consultar o obtener información del servidor.

post se utiliza para enviar información al servidor, por ejemplo, cuando queremos registrar un nuevo trabajador en la base de datos.

19. ¿Por qué se debe validar la información antes de almacenarla?

porque permite comprobar que los datos recibidos sean correctos y estén completos antes de guardarlos.

por ejemplo, podemos verificar que el nombre no esté vacío, que el teléfono tenga un formato válido y que los campos obligatorios hayan sido enviados. esto ayuda a evitar información incorrecta en la base de datos.

20. ¿Qué sucede cuando la API intenta insertar un registro en una tabla que no existe?

mysql genera un error porque la tabla indicada en la consulta no existe.

la api debe detectar ese error y enviar una respuesta adecuada al cliente, en lugar de indicar que el registro se guardó correctamente.

21. ¿Por qué es importante distinguir entre una respuesta exitosa y una respuesta de error?

porque permite que el frontend sepa qué ocurrió con la solicitud.

por ejemplo, una respuesta 200 puede indicar que una consulta fue exitosa, mientras que una respuesta 400 puede indicar que los datos enviados son incorrectos y una 500 puede indicar un problema interno del servidor.

22. ¿Qué diferencia existe entre los errores de conexión y los errores de consulta SQL?

un error de conexión ocurre cuando la api no puede comunicarse con el servidor de mysql. puede deberse a datos de conexión incorrectos, problemas de red o que el servidor no esté disponible.

un error de consulta sql ocurre cuando sí existe una conexión con mysql, pero la instrucción sql tiene algún problema, por ejemplo, intentar insertar datos en una tabla que no existe.

23. ¿Por qué se deben utilizar consultas parametrizadas?

porque permiten enviar los datos separados de la instrucción sql. esto ayuda a evitar que un usuario pueda introducir código sql malicioso dentro de los datos enviados.

por esta razón, las consultas parametrizadas ayudan a proteger la base de datos contra ataques de inyección sql

24. ¿Qué función cumple CORS cuando la API se consume desde Angular?

cors permite que una aplicación de angular pueda realizar solicitudes a la api aunque estén ejecutándose en diferentes orígenes o puertos.

por ejemplo, angular puede ejecutarse en un puerto y la api de node.js en otro. cors permite que el navegador autorice esa comunicación.

25. ¿Qué ventajas ofrece almacenar la información en una base de datos en línea?

permite que la información pueda almacenarse y consultarse desde diferentes lugares siempre que exista una conexión a internet y los permisos necesarios.

también facilita que una aplicación pueda trabajar con información centralizada y que los datos puedan mantenerse disponibles para diferentes usuarios o aplicaciones.

26. ¿Por qué es importante probar los casos incorrectos y no solamente los exitosos?

porque una aplicación debe saber cómo reaccionar cuando el usuario introduce información incorrecta o cuando ocurre algún problema.

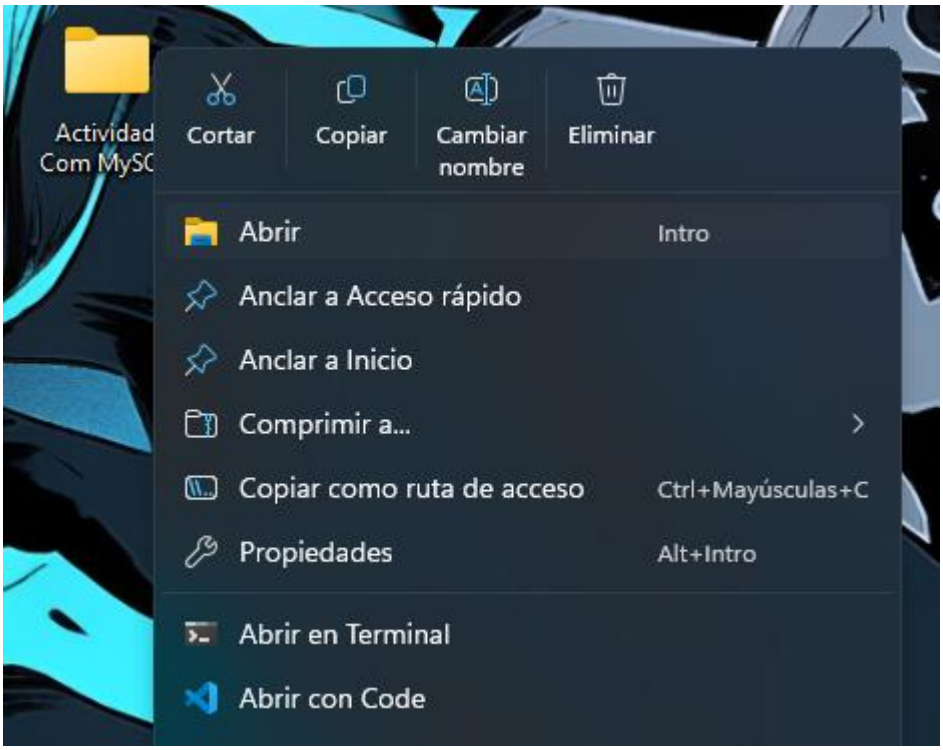
27. ¿Cómo se puede proteger la información de conexión a la base de datos?

una forma recomendada es utilizar variables de entorno para guardar datos como el usuario, contraseña, host y nombre de la base de datos.

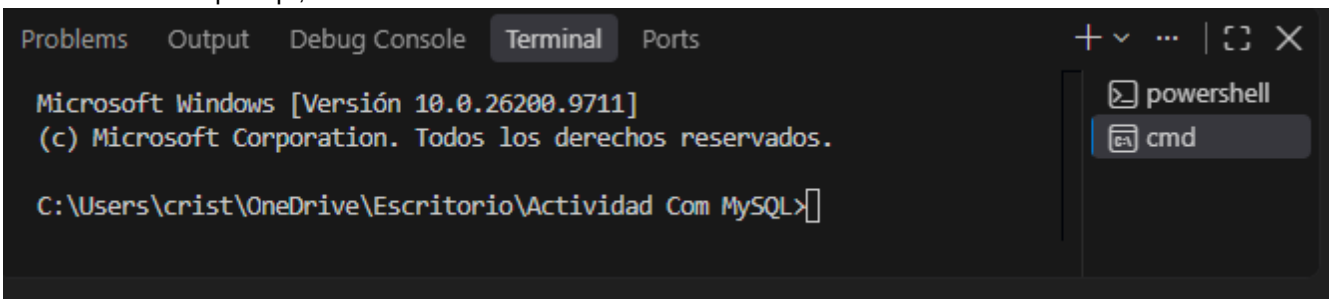
también se debe evitar escribir directamente estas credenciales en el código que se sube a un repositorio público. de esta manera se reduce el riesgo de que otras personas obtengan acceso a la base de datos.

Registro sobre toda la instalacion y paso a paso

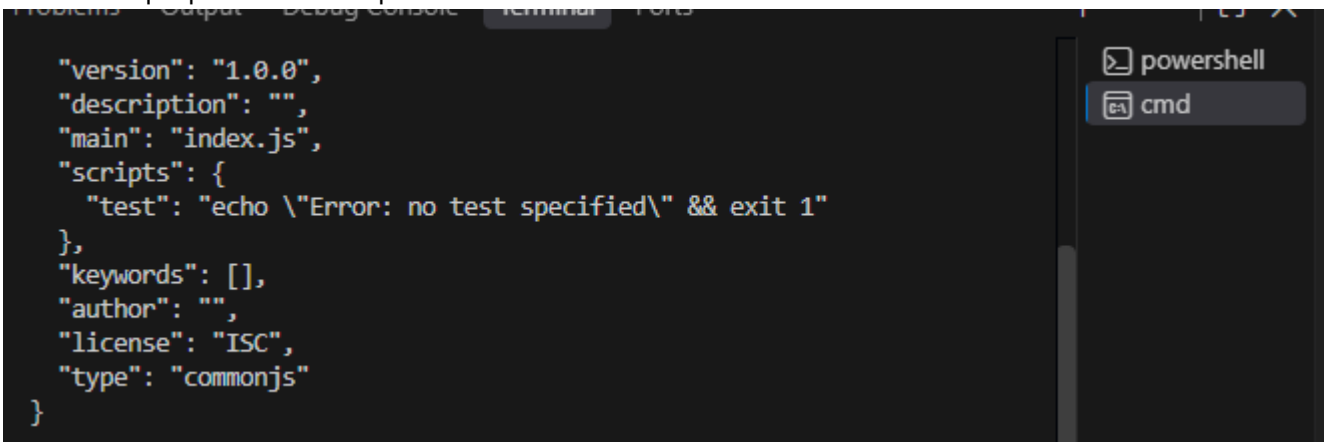
El primer paso es crear una carpeta y ponerle el nombre de nuestra preferencia, después entramos en ella y abrimos VSCode



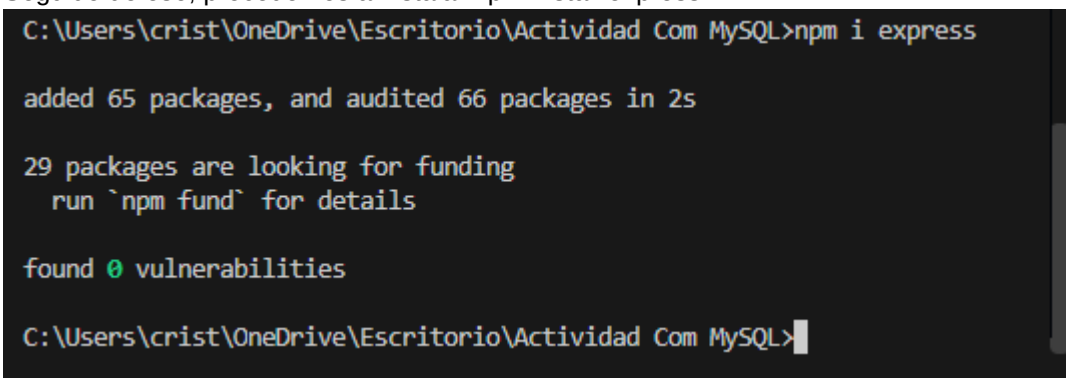
Una vez estando dentro le damos control + ñ. Que abrirá la terminal para posteriormente al lado de powershell, buscar command prompt, seleccionamos



Y una vez estando dentro proseguimos a colocar los comandos para inicializar el proyecto, primero escribimos npm init -y . Y damos enter, eso lo que hará es crearnos el archivo package.json que es donde mas adelante tendremos que poner cierto script



Seguido de eso, procedemos a instalar npm install express



Después instalamos el npm install --save-dev nodemon

```
C:\Users\crist\OneDrive\Escritorio\Actividad Com MySQL>npm i --save-dev n
odemon

added 26 packages, and audited 92 packages in 2s

34 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\crist\OneDrive\Escritorio\Actividad Com MySQL>
```

Y seguido instalamos el cors npm install cors

```
C:\Users\crist\OneDrive\Escritorio\Actividad Com MySQL>npm i cors

added 2 packages, and audited 94 packages in 914ms

35 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\crist\OneDrive\Escritorio\Actividad Com MySQL>
```

Por ultimo instalamos el npm install mysql2 para poder conectarnos y trabajar con la base de datos de MySQL

```
C:\Users\crist\OneDrive\Escritorio\Actividad Com MySQL>npm i mysql2

added 10 packages, and audited 104 packages in 2s

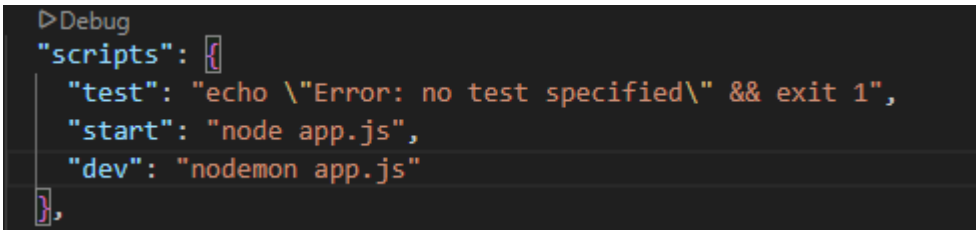
37 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\crist\OneDrive\Escritorio\Actividad Com MySQL>
```

Después vamos al archivo ya creado package.json que es donde tendremos que poner el script: “start”:”node ‘NOMBRE DE NUESTRO JS’”,

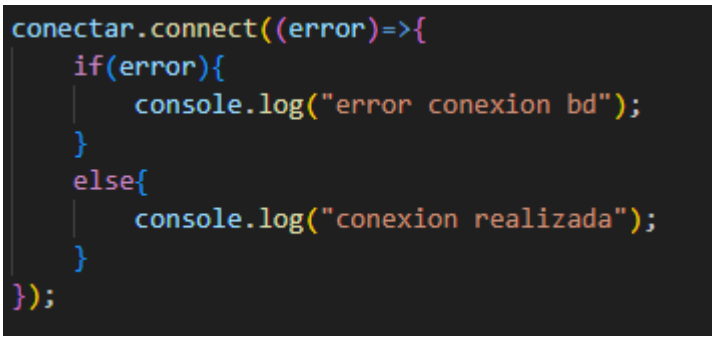
“dev”:”’NOMBRE DE NUESTRO JS’”



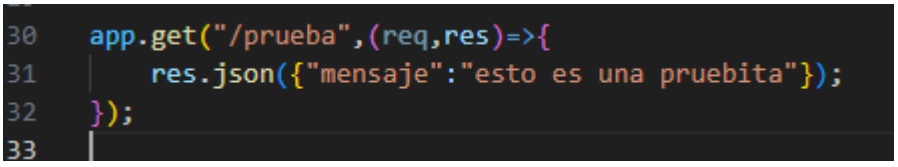
Aproximacion comenzare con las capturas del codigo, en esta captura se importan las dependencias necesarioas para el proyecto: Express para cerear el servidor, Cors para permitir conexiones externas y Mysql2 para cibectar la aplicación con MySQL. Tambien se configura el puerto 1350 y los middleware necesarios



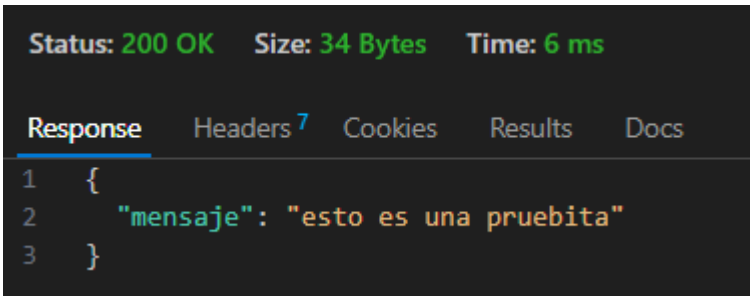
En esta parte se configura la conexión con la base de datos construccion de MySQL. Se indican el servidor, usuario, contraseña, nombre de la base de datos y puerto. Despues se verifica si la conexión fue realizada correctamente



En esta parte se crea una ruta GET llamada /prueba. Su función es comprobar que el servidor esta funcionando y que puede responder correctamente a una petición

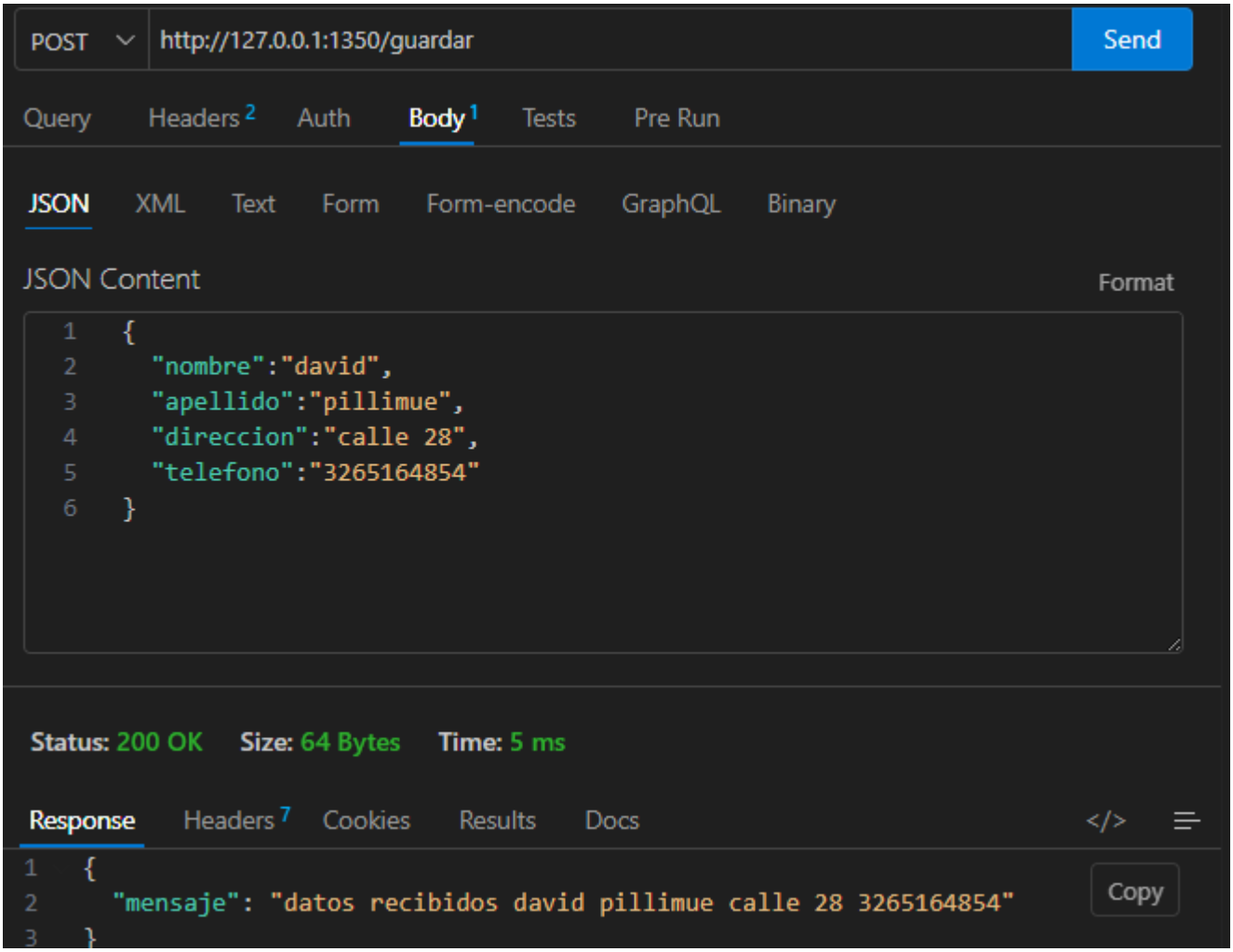


```
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Servidor en ejecucion 1350
conexion realizada
```



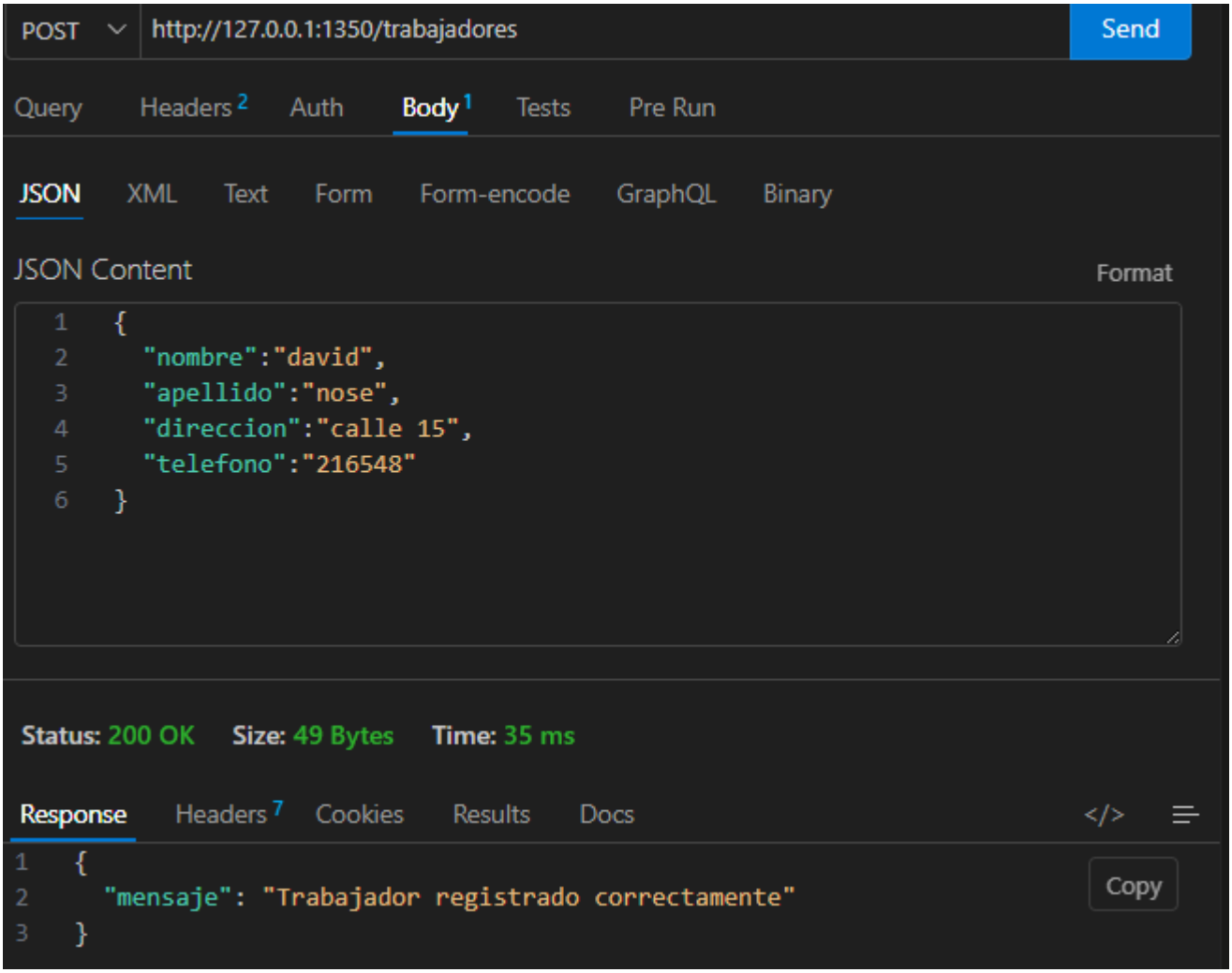
En esta parte se crea una ruta POST para recibir los datos de un trabajador. La información se obtiene mediante req.body y posteriormente se devuelve una respuesta mostrando los datos recibidos

```
34 app.post("/guardar",(req,res)=>{
35     const {nombre,apellido,direccion,telefono}=req.body;
36
37     res.json({
38         "mensaje":"datos recibidos "+nombre+" "+apellido+" "+direccion+" "+telefono
39     });
40 });
```



En esta parte se crea la ruta POST /trabajadores, que permite registrar un trabajador en la tabla trabajadores. Los datos recibidos se envían mediante una consulta SQL utilizando parámetros para insertar la información en la base de datos

```
app.post("/trabajadores",(req,res)=>{
    const {nombre,apellido,direccion,telefono}=req.body;
    conectar.query("INSERT INTO trabajadores (nombre,apellido,direccion,telefono) VALUES (?,?,?,?)",
        [nombre,apellido,direccion,telefono],(error,resultado)=>{
            if(error){
                return res.json({"mensaje":"Error al registrar el trabajador"});
            }
            else{
                return res.json({"mensaje":"Trabajador registrado correctamente"});
            }
        });
});
```



En esta ultima parte se crea una ruta GET para consultar todos los trabajadores almacenados en la base de datos. Finalmente, app.listen inicia el servidor en el puerto 1350 para poder realizar las peticiones a la API

```
app.get("/trabajadores",(req,res)=>{
    conectar.query("SELECT * FROM trabajadores",(error,resultados)=>{
        if(error){
            return res.json({"mensaje":"Error al traer los trabajadores"});
        }
        else{
            return res.json(resultados);
        }
    });
});

app.listen(PORT,()=>{
    console.log("Servidor en ejecucion "+PORT);
});
```

GET http://127.0.0.1:1350/trabajadores Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "nombre":"david",
  "apellido":"nose",
  "direccion":"calle 15",
  "telefono":"216548"
}
```

Status: 200 OK Size: 280 Bytes Time: 6 ms

Response Headers 7 Cookies Results Docs </>

Copy

```
  },
  {
    "id": 2,
    "nombre": "david",
    "apellido": "pillimue",
    "direccion": "calle 28",
    "telefono": "3265164854"
  },
  {
    "id": 3,
    "nombre": "david",
    "apellido": "nose",
    "direccion": "calle 15",
    "telefono": "216548"
  }
]
```

id	nombre	apellido	direccion	telefono
1	cristian	yalanda	calle 65	3215875686
2	david	pillimue	calle 28	3265164854
3	david	nose	calle 15	216548
NULL	NULL	NULL	NULL	NULL

Y en esta parte es cuando se usa para la conexión de la base de datos de workbench

```
let conectar=mysql.createConnection({
    host:"127.0.0.1",
    user:"root",
    password:"yalanda15963",
    database:"construccion",
    port:3306
});
```

Esta es la información que tengo en mi base de datos de workbench

```
create database construccion;
use construccion;

create table trabajadores(
    id int auto_increment primary key,
    nombre varchar(50) not null,
    apellido varchar(50) not null,
    direccion varchar(50) not null,
    telefono varchar(15) not null
);
```

5. Criterios de evaluación

Criterio	Puntaje
Investigación y preparación del entorno	10
Diseño de la base de datos	15
Construcción y funcionamiento de la API	25
Identificación, explicación y corrección de errores	25
Pruebas funcionales y evidencias	15
Documentación y repositorio	10
TOTAL	100 puntos